

# Main Branch Complete Changes & Code Architecture Report

**Scope:** Complete code-level documentation of all features, PWA transparent branding, dish pairing/upsell engine, real-time sync, and database cache invalidations pushed & merged to main.

|                                                                                        |                                                                 |
|----------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Target Branch:</b> main (Merged from Menu-Suggestions)                              | <b>PR &amp; Merge Commit:</b> PR #5 (Commit de0ecd8)            |
| <b>Tech Stack:</b> Next.js 14 App Router, TypeScript, Prisma, Tailwind CSS, WebSockets | <b>Status:</b> Production Ready & Verified on Local Environment |

## 1. Executive Summary & Overview of Work

All changes requested during the session have been cleanly implemented, tested, pushed to origin Menu-Suggestions, and merged into main via PR #5. The local workspace has been refreshed and is on a clean main branch.

- **PWA Brand Identity & Splash Screen Fix:** Converted logo to transparent background, generated all standardized PWA icons (512x512, 192x192, apple-touch-icon, favicon), and resolved manifest collision.
- **Admin & Kitchen Pairing / Suggestion Management:** Added full edit/save controls for best\_pairing in both Admin and Kitchen portals with 8 quick-preset chips.
- **Customer-Facing Suggestion Engine & Upsell Modals:** Prominent badge on dish cards and dynamic companion recommendation cards inside the item detail popup.
- **Cache Invalidation & Oms Real-Time Sync:** Implemented explicit Next.js revalidateTag('menu-catalog') and WebSocket broadcasts for instantaneous updates.

## 2. PWA Icon & Splash Screen Refactor

**Problem:** On mobile devices (Android & iOS) when adding the app to Home Screen, a dark circular badge background artifact appeared during splash loading.

**Root Cause:** Android Chrome builds splash screens dynamically from the manifest's icons property and background\_color. The previous PNG had an embedded off-color backdrop and a Next.js dynamic route clash existed between manifest.ts and manifest.json.

| File Modified / Created                                             | Technical Action Taken                                                                               |
|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| apps/web/public/logo-transparent.png                                | Created transparent alpha-masked PNG from original brand logo without rectangular backdrop.          |
| apps/web/public/icon-512.png<br>apps/web/public/icon-192.png        | Centered logo on transparent canvas (512x512 & 192x192) with purpose any maskable.                   |
| apps/web/public/apple-touch-icon.png<br>apps/web/public/favicon.ico | Crisp 180x180 iOS touch icon and multi-resolution standard favicon.                                  |
| apps/web/public/manifest.json<br>apps/web/app/layout.tsx            | Removed duplicate manifest.ts. Fixed theme_color: #FFFBEB and background_color: #FFFBEB in manifest. |

### 3. Dish Suggestions & Pairings Architecture

The system now supports bidirectional management and customer-facing discovery of food-and-beverage pairings (e.g. *Triple Decker Club Sandwich* → *Smol Chai* / *Classic Cold Coffee*).

- **1. Database (PostgreSQL / SQLite via Prisma):** MenuItem model contains best\_pairing String? column.
- **2. Server Actions (menu-actions.ts):** fetchAdminMenuCatalogAction() maps best\_pairing into AdminMenuItem. saveMenuItemAction() writes to DB and triggers revalidateTag('menu-catalog').
- **3. Admin / Kitchen Portal:** Staff can type custom pairings or click quick-chips (e.g. + Smol Chai, + Mint Mojito) and click 'Save Item'.
- **4. Real-Time Broadcast:** On save, a WebSocket event is fired to connected clients updating the client-side state without page refresh.
- **5. Customer Card & Modal:** Prominent badge ■ Pairs with: [Item] is rendered on dish cards. Tapping the dish opens the Detail Modal which tokenizes the pairing string and renders direct + Add to Cart quick-buy cards.

## 4. Code Walkthrough: Backend & Server Actions

**Files:** `apps/web/app/admin/menu-actions.ts` & `apps/web/app/kitchen/menu-actions.ts`

We updated the TypeScript interfaces and server actions to include `bestPairing`, and added Next.js cache revalidation tags to prevent stale menu data across devices.

```
// Interface Definition & Payload
export interface SaveMenuItemPayload {
  id?: string;
  name: string;
  price: number;
  category: string;
  description?: string;
  bestPairing?: string; // Newly added field
  imageUrl?: string;
  isAvailable?: boolean;
}

// Server Action with Next.js Cache Revalidation
export async function saveMenuItemAction(payload: SaveMenuItemPayload) {
  const result = await saveMenuItemToDb({
    ...payload,
    best_pairing: payload.bestPairing?.trim() || null,
  });

  // Invalidate cached menu catalog across all routes instantly
  revalidateTag("menu-catalog");
  revalidatePath("/menu");
  revalidatePath("/admin/menu");
  revalidatePath("/kitchen/menu");
  return result;
}
```

## 5. Code Walkthrough: Admin & Kitchen Management UI

**File:** `apps/web/components/admin/AdminMenuManager.tsx`

Added pairing state management, quick preset chips, and card badge indicators so managers can effortlessly configure pairings with single clicks.

```
// Quick Suggestion Chips Component in Admin Edit Modal
<div className="space-y-1.5">
  <label className="text-xs font-semibold text-stone-700 flex items-center justify-between">
    <span>Best Pairing / Chef Suggestion</span>
    <span className="text-[10px] text-amber-700 font-normal">Optional</span>
  </label>
  <input
    type="text"
    placeholder="e.g. Smol Chai / Classic Cold Coffee"
    value={formBestPairing}
    onChange={(e) => setFormBestPairing(e.target.value)}
    className="w-full text-xs rounded-lg border border-stone-300 p-2 focus:ring-1 focus:ring-amber-500"
  />
  {/} One-click quick presets {/}
  <div className="flex flex-wrap gap-1.5 pt-1">
    {["Smol Chai", "Cold Coffee", "Mint Mojito", "Triple Decker", "Peri Peri Fries"].map((chip) => (
      <button
        key={chip}
        type="button"
        onClick={() => setFormBestPairing(prev => prev ? `${prev} / ${chip}` : chip)}
        className="text-[10px] px-2 py-0.5 rounded-full bg-amber-50 text-amber-800 border border-amber-200
        hover:bg-amber-100"
      >
        + {chip}
      </button>
    ))}
  </div>
</div>
```

## 6. Code Walkthrough: Customer UI & Upsell Modals

**Files:** `apps/web/components/menu/MenuItemCard.tsx` & `apps/web/components/menu/ItemDetailModal.tsx`

The customer menu displays a warm amber badge on dish cards and parses pairing tokens inside the detail modal to recommend matchable menu items with an instant 'Add' button.

```
// ItemDetailModal: Multi-Item Pairing Matching Logic
const pairingTokens = useMemo(() => {
  if (!item.best_pairing) return [];
  return item.best_pairing
    .split(/[,]\b|\band\b/i)
    .map(t => t.trim().toLowerCase())
    .filter(Boolean);
}, [item.best_pairing]);

// Find matching items from full catalog
const matchedPairings = useMemo(() => {
  if (!pairingTokens.length) return [];
  return catalog.filter(catalogItem => {
    if (catalogItem.id === item.id) return false;
    const catName = catalogItem.name.toLowerCase();
    return pairingTokens.some(token => catName.includes(token) || token.includes(catName));
  }).slice(0, 3);
}, [pairingTokens, catalog, item.id]);

// Render Companion Recommendation Card
{matchedPairings.map((pairingItem) => (
  <div key={pairingItem.id} className="flex items-center justify-between p-2 rounded-xl bg-amber-50/70 border border-amber-200/80">
    <div className="flex items-center gap-2">
      <span className="text-xs font-semibold text-stone-800">{pairingItem.name}</span>
      <span className="text-xs text-amber-900 font-bold">■{pairingItem.price}</span>
    </div>
    <button onClick={() => addItemToCart(pairingItem)} className="px-2.5 py-1 text-xs bg-amber-600 text-white rounded-lg font-medium hover:bg-amber-700">
      + Add
    </button>
  </div>
))}
```

## 7. Step-by-Step Testing & Verification Guide

- **Step 1 — Start Dev Server:** Run `npm run dev` from repository root.
- **Step 2 — Open Admin Portal:** Navigate to `http://localhost:3000/admin/menu`. Click 'Edit' on any dish (e.g. *Triple Decker Club Sandwich*).
- **Step 3 — Modify Pairing:** Click chip + Smol Chai or type custom text. Click 'Save Item'. Verify toast notification appears.
- **Step 4 — Customer Menu Verification:** Navigate to `http://localhost:3000/menu`. Verify the dish card immediately shows the badge ■ pairs with: Smol Chai without page refresh.
- **Step 5 — Upsell Modal Verification:** Click the dish card to open the detail popup. Verify 'Pairs beautifully with' section displays *Smol Chai* with a direct + Add button.
- **Step 6 — PWA Mobile Splash Check:** Open in mobile Chrome/Safari, tap 'Add to Home screen', launch the app, and verify the splash screen has a pristine transparent logo on warm cream background.

## 8. Git Commits & Merge Status

| Commit / PR              | Branch           | Description                                                                         |
|--------------------------|------------------|-------------------------------------------------------------------------------------|
| PR #5<br>(Merge de0ecd8) | main             | Merged branch Menu-Suggestions into main with all features.                         |
| e387c95                  | Menu-Suggestions | Added suggestions/pairings in Admin & Kitchen, real-time sync, and customer badges. |
| b9cda44                  | Menu-Suggestions | Transparent PWA splash screen icons, apple-touch-icon, and manifest fixes.          |